MeanFlow, hacked on a 2D checkerboard

Hacker role, CS 2883R, 14 Oct. Paper: Geng, Deng, Bai, Kolter, He, *Mean Flows for One-step Generative Modeling* (arXiv 2505.13447).

Everything is in `meanflow.py`, written from Alg. 1 and Alg. 2 of the paper. Five models, all identical except for one knob. Shared settings are the paper's CIFAR-10 ones (App. A): logit-normal(−2, 2) time sampler, adaptive loss power p = 0.75. The net is an MLP with 3 hidden layers of 512, trained for 20k steps.

model	knob
`fm`	ratio of r≠t = 0, which is exactly Flow Matching
`mf`	ratio = 75%, JVP tangent (v, 0, 1): MeanFlow
`tangent_v00`, `_v10`, `_v11`	MeanFlow with a wrong JVP tangent (paper Tab. 1b)

Weights are in `weights/`. Delete a file to retrain that model (about 10 min on a laptop CPU).

In [1]:
```
import copy, inspect, warnings, torch, matplotlib.pyplot as plt
warnings.filterwarnings("ignore")
from meanflow import *

torch.manual_seed(0)
data = checkerboard(20000)
z1 = torch.randn(20000, 2, generator=torch.Generator().manual_seed(7))  # same noise for every model
nets = {k: load_or_train(k) for k in VARIANTS}
```

The core: Alg. 1 in PyTorch

In [2]:
```
print(inspect.getsource(loss_fn))
```

```
def loss_fn(net, x, ratio=0.75, p=0.75, c=1e-3, tangent=(1, 0, 1), mu=-2.0,
sigma=2.0):
    """Alg. 1. Defaults are the paper's CIFAR-10 setting (App. A); its Image
Net one (25%, lognorm(-0.4, 1), p = 1)
    gave a blurrier 1-step result on this toy. `tangent` scales the (v, dr,
dt) JVP direction; (1, 0, 1) is correct, others are Tab. 1b ablations."""
    t, r = sample_t_r(len(x), ratio, mu, sigma)
    e = torch.randn_like(x)
    z = (1 - t) * x + t * e
    v = e - x
    kv, kr, kt = tangent
    u, dudt = jvp(net, (z, r, t), (kv * v, kr * torch.ones_like(r), kt * tor
ch.ones_like(t)))
    u_tgt = v - (t - r) * dudt
    sq = (u - u_tgt.detach()).pow(2).sum(dim=1)
    w = 1 / (sq.detach() + c) ** p  # adaptive weight (Tab. 1e)
    return (w * sq).mean()
```

Sanity check: `jvp` along (v, 0, 1) is the total derivative du/dt. Compare it to a finite difference that moves z and t together.

In [3]:
```
net = copy.deepcopy(nets["mf"]).double()  # float64 so the finite differenc
e is trustworthy
z, r, t, v = (torch.randn(512, 2), torch.rand(512, 1) * 0.4, 0.5 + torch.ra
nd(512, 1) * 0.5, torch.randn(512, 2))
z, r, t, v = (a.double() for a in (z, r, t, v))
_, dudt = torch.func.jvp(net, (z, r, t), (v, torch.zeros_like(r), torch.one
s_like(t)))
h = 1e-5
fd = (net(z + h * v, r, t + h) - net(z - h * v, r, t - h)) / (2 * h)
print(f"max |jvp - finite diff| = {(dudt - fd).abs().max():.1e}   (scale of
dudt: {dudt.abs().mean():.2f})")
```

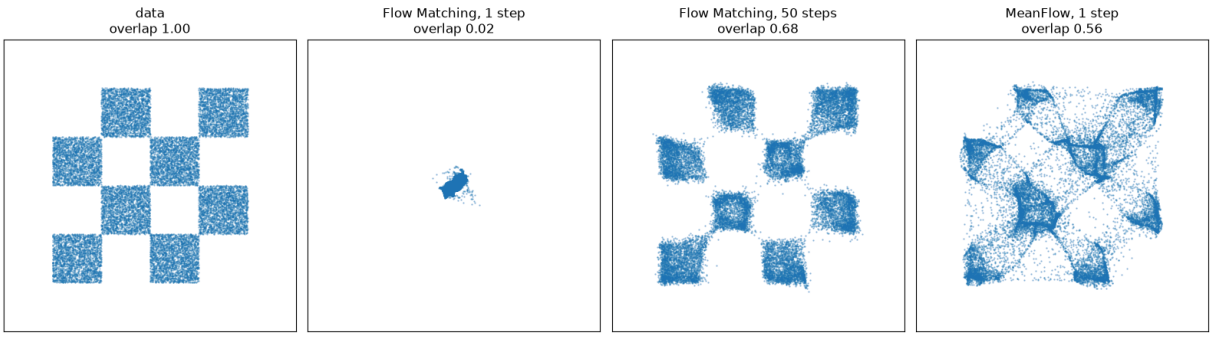
```
max |jvp - finite diff| = 2.1e-06   (scale of dudt: 1.84)
```

1 step: Flow Matching vs MeanFlow

In [4]:
```
def show(ax, x, title):
    ax.scatter(*x.T, s=0.3, alpha=0.5)
    ax.set(xlim=(-3, 3), ylim=(-3, 3), aspect="equal", xticks=[], yticks=
[])
    ax.set_title(f"{title}\noverlap {overlap(x, data):.2f}")

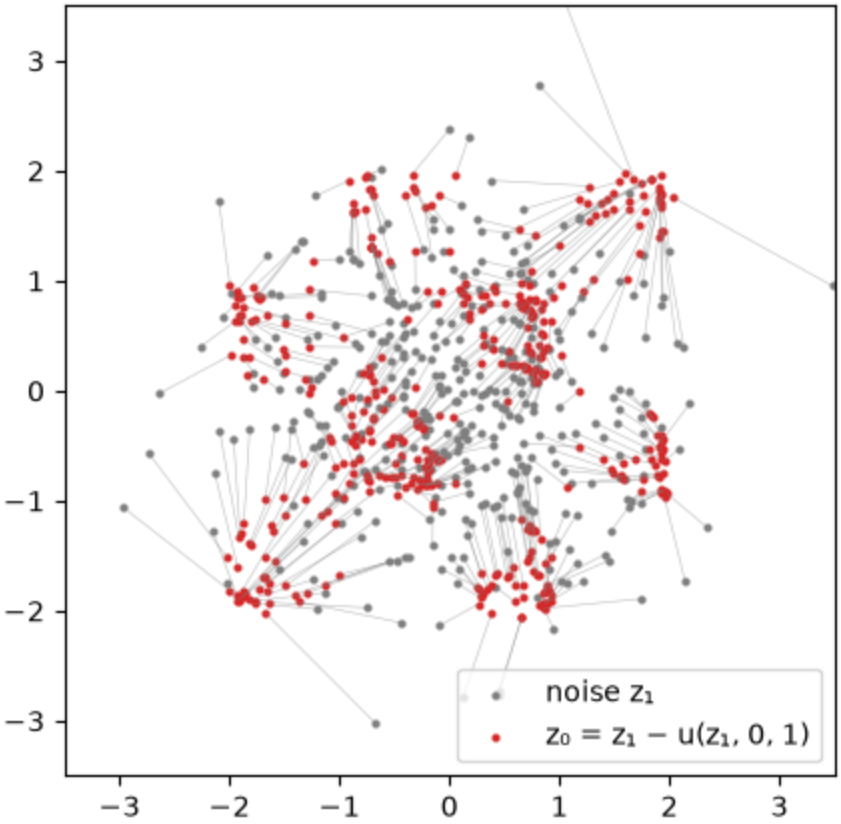
panels = [
    (data, "data"),
    (sample(nets["fm"], 20000, 1, instantaneous=True, z=z1), "Flow Matchin
g, 1 step"),
    (sample(nets["fm"], 20000, 50, instantaneous=True, z=z1), "Flow Matchin
g, 50 steps"),
    (sample(nets["mf"], 20000, 1, z=z1), "MeanFlow, 1 step"),
]
fig, axes = plt.subplots(1, 4, figsize=(16, 4.4))
for ax, (x, title) in zip(axes, panels):
```

```
    show(ax, x, title)
plt.tight_layout()
```



Flow Matching's single step lands on the data mean: at t = 1 the noise says nothing about x, so v(z, 1) = z − E[x]. MeanFlow's u(z, 0, 1) is the average velocity of the whole path, so one step jumps straight to the data. The lines below go from noise to 1-step sample.

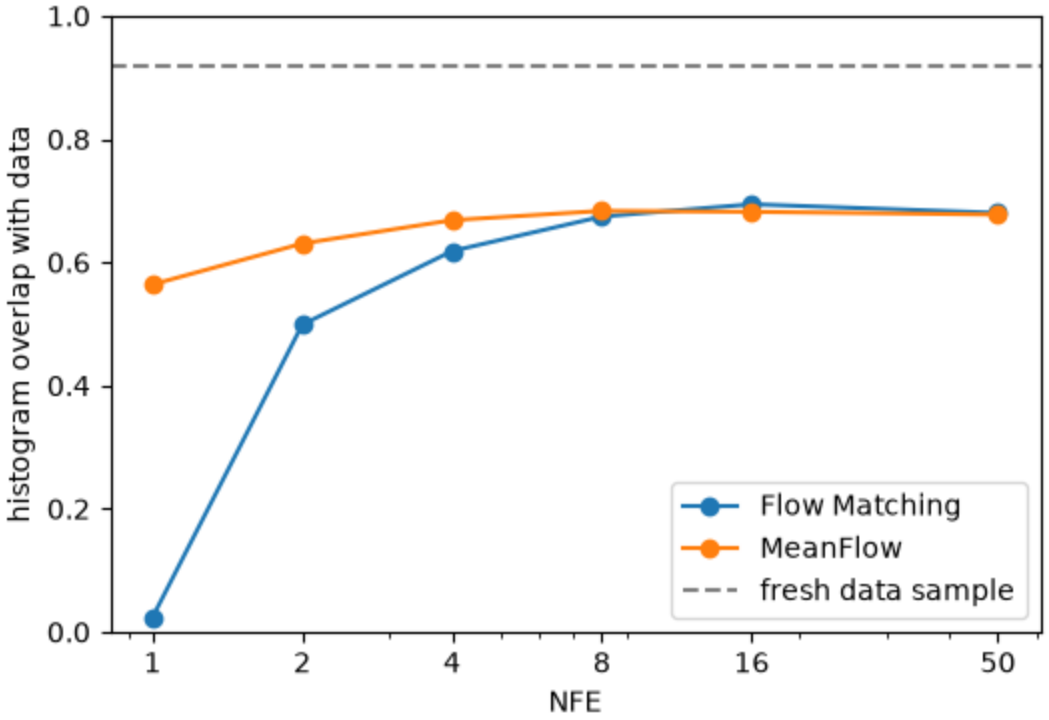
In [5]:
```
x0 = sample(nets["mf"], 400, 1, z=z1[:400])
fig, ax = plt.subplots(figsize=(5, 5))
for a, b in zip(z1[:400], x0):
    ax.plot([a[0], b[0]], [a[1], b[1]], lw=0.4, c="gray", alpha=0.5)
ax.scatter(*z1[:400].T, s=4, c="tab:gray", label="noise z₁")
ax.scatter(*x0.T, s=4, c="tab:red", label="z₀ = z₁ − u(z₁, 0, 1)")
ax.set(xlim=(-3.5, 3.5), ylim=(-3.5, 3.5), aspect="equal"); ax.legend(loc
="lower right");
```



Quality vs number of steps

In [6]:
```
nfes = [1, 2, 4, 8, 16, 50]
curves = {
    "Flow Matching": [overlap(sample(nets["fm"], 20000, n, instantaneous=Tr
ue, z=z1), data) for n in nfes],
    "MeanFlow": [overlap(sample(nets["mf"], 20000, n, z=z1), data) for n in
nfes],
}
floor = overlap(checkerboard(20000), data)
fig, ax = plt.subplots(figsize=(6, 4))
for k, c in curves.items():
    ax.plot(nfes, c, "o-", label=k)
ax.axhline(floor, ls="--", c="gray", label="fresh data sample")
ax.set(xscale="log", xticks=nfes, xticklabels=nfes, xlabel="NFE", ylabel="h
istogram overlap with data", ylim=(0, 1))
ax.legend()
for k, c in curves.items():
    print(k.ljust(14), "  ".join(f"{n}:{o:.2f}" for n, o in zip(nfes, c)))
print("data vs data  ", f"{floor:.2f}")
```

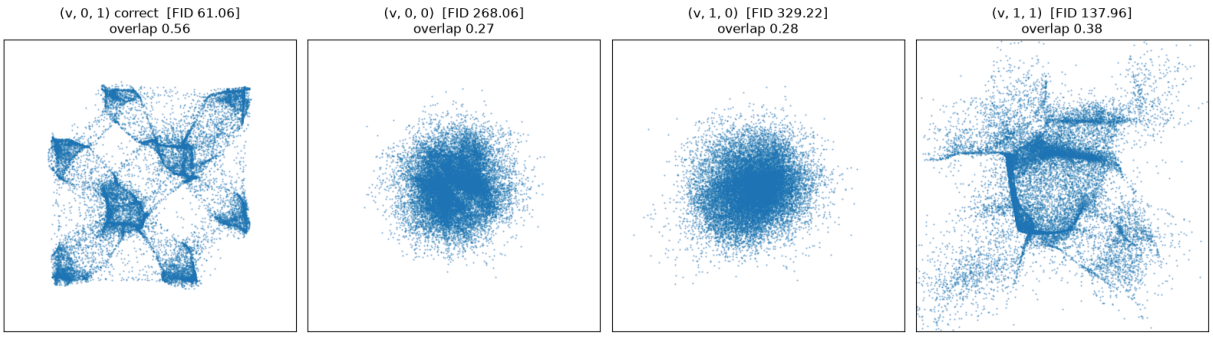
```
Flow Matching  1:0.02  2:0.50  4:0.62  8:0.67  16:0.69  50:0.68
MeanFlow       1:0.56  2:0.63  4:0.67  8:0.68  16:0.68  50:0.68
data vs data   0.92
```



Break the JVP (paper Tab. 1b)

Same training, but the tangent for (∂z u, ∂r u, ∂t u) is wrong. Paper FIDs (1-NFE, ImageNet, B/4) in brackets.

In [7]:
```
paper = {"mf": 61.06, "tangent_v00": 268.06, "tangent_v10": 329.22, "tangen
t_v11": 137.96}
names = {"mf": "(v, 0, 1) correct", "tangent_v00": "(v, 0, 0)", "tangent_v1
0": "(v, 1, 0)", "tangent_v11": "(v, 1, 1)"}
fig, axes = plt.subplots(1, 4, figsize=(16, 4.4))
for ax, k in zip(axes, paper):
    show(ax, sample(nets[k], 20000, 1, z=z1), f"{names[k]}  [FID {paper
[k]}]")
plt.tight_layout()
```



Same ordering as the paper: correct ≫ (v, 1, 1) > (v, 0, 0) ≈ (v, 1, 0). Without $\partial_t u$ the network never learns that u should change with the interval, so the one big jump barely moves the noise.

What I learned hacking it

1. The whole method is one changed target: `u_tgt = v − (t − r) ∗ dudt`. With r = t it is Flow Matching.
2. Settings: the paper's ImageNet defaults (25% r≠t, lognorm(−0.4, 1), p = 1) gave a smeared 1-step checkerboard on this toy. Its CIFAR-10 settings (75%, lognorm(−2, 2), p = 0.75) gave clean squares.
3. The time embedding matters more than I expected. With DiT-style frequencies (up to ~1000), $\partial_t u$ is huge and training diverged. Low-frequency sin/cos of t fixed it.
4. My first metric (sliced Wasserstein) ranked a smeared 1-step sample above a clean 50-step one, so look at the pictures.

Credits: method and pseudocode from the paper (Alg. 1, 2; Tab. 1, 4). Code written from scratch in PyTorch (`torch.func.jvp`). The authors implemented it in JAX (paper §4.1); I did not use any existing MeanFlow code.